

Multi-Agent Embodied Architecture: multi_brains

Multi-Agent Embodied Architecture: multi_brains

1. Course Content
2. Introduction to multi_brains
3. Dual-Model Inference Architecture
 - 3.1 Advantages of Dual-Model Inference
 - 3.1.1 Decoupling Task Decision and Action Conversion
 - 3.1.2 Improving the Success Rate of Long-Process Tasks
 - 3.2 Decision Layer AI
 - 3.3 Execution Layer AI
4. Task Cycle
5. Historical Context
6. Map Mapping
7. Action Function Library

1. Course Content

Understand the multi_brains framework, the multi-agent embodied architecture of ROSMASTER-M3.

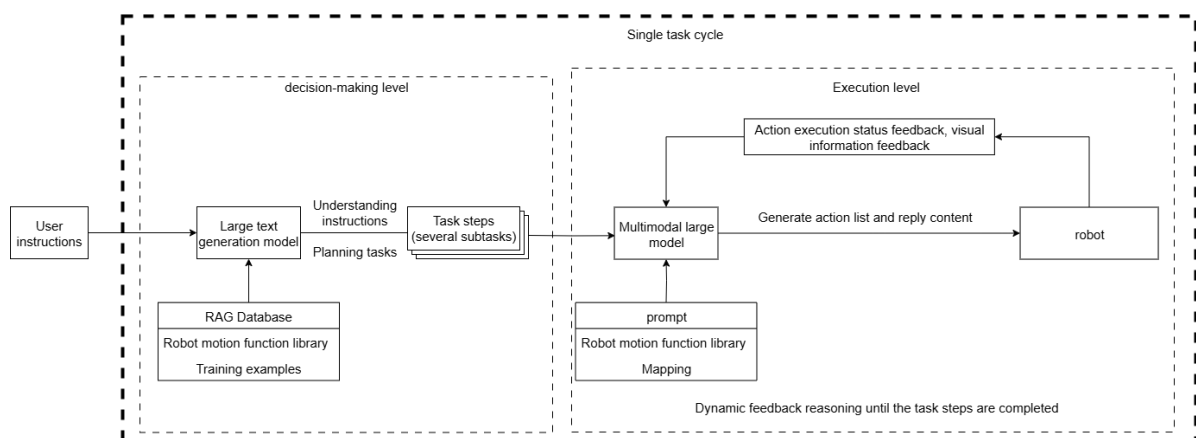
2. Introduction to multi_brains

- multi_brains is an embodied intelligence framework developed by Yahboom, designed to adapt to general-purpose AI large models. It avoids the issues of VLA and end-to-end models, which require dedicated hardware, local computing power support, and long data collection and training times. By simply accessing general AI models from cloud providers, edge robots can achieve powerful embodied intelligence effects.
- multi_brains uses an RAG knowledge base to expand training samples, allowing the robot to quickly adapt to different scenario tasks. It can be deployed on most small-scale main control devices, with the massive model inference handled in the cloud or on a local server.
- The core idea of multi_brains is to use multiple AI models to decouple different stages of a task in a layered manner. It is a comprehensive improvement over the first-generation, self-developed dual-model inference framework. Its main enhancements include:
 - Smoother recording experience with inertial tail-end detection and adjustable detection duration.
 - More flexible decision-making mechanism with an added task routing function. The AI agent can autonomously choose between dual-model and single-model inference based on task difficulty, giving the AI greater autonomy and a better user experience.
 - Easy-to-understand visualized architecture flow. multi_brains uses Dify to build visual agents, allowing real-time observation of the AI agent's operation and data flow during debugging, giving users a more intuitive understanding of the dual-model inference process.
 - Fully localized RAG knowledge base, eliminating concerns about data sensitivity.

- Seamless integration with most AI models worldwide. Through Dify's model provider plugins, hundreds of AI models from around the globe can be quickly and seamlessly integrated (local models can also be connected if you have local deployment needs).
- Traceable history. By querying Dify's backend access history, users can look up all task commands and the history of steps executed by the robot.
- Personalized voice responses. Users can customize random response voices. Configuration of built-in speech synthesis commands is simple and user-friendly.
- A complete testing toolbox for quick verification of core functional modules.
- Simpler getting-started steps, with most operations simplified and shortcut commands configured.

3. Dual-Model Inference Architecture

- The design of **dual-model inference** and **dynamic feedback inference** provides stronger system robustness compared to a single-model architecture.



3.1 Advantages of Dual-Model Inference

3.1.1 Decoupling Task Decision and Action Conversion

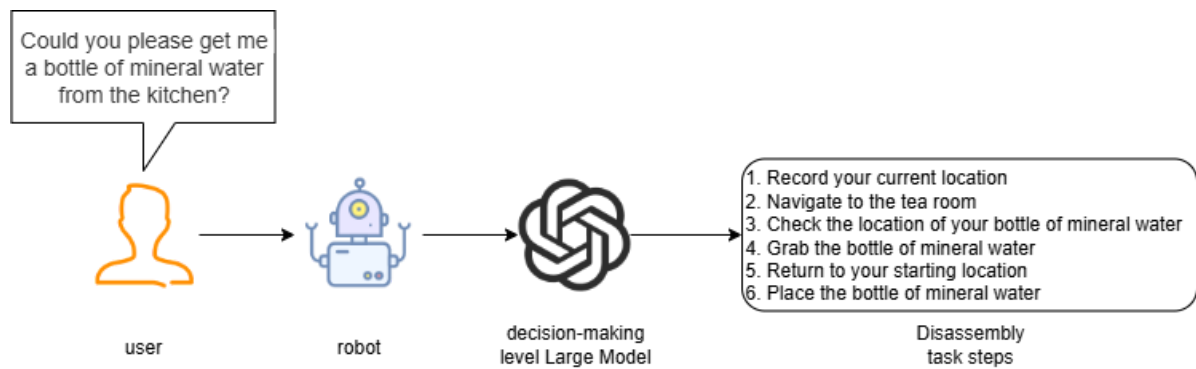
- The Decision Layer AI focuses on thinking, planning, and breaking down task commands.
- The Execution Layer AI focuses on chat responses and converting task steps into JSON text.

3.1.2 Improving the Success Rate of Long-Process Tasks

When using a single model for inference, it needs to perform "natural language understanding + environmental perception + process decomposition + action function output" simultaneously, which can easily lead to **modal interference** (misunderstanding due to language ambiguity or overly long commands). The dual-model approach handles this in stages, allowing the decision layer to focus on task planning and the execution layer to focus on action execution.

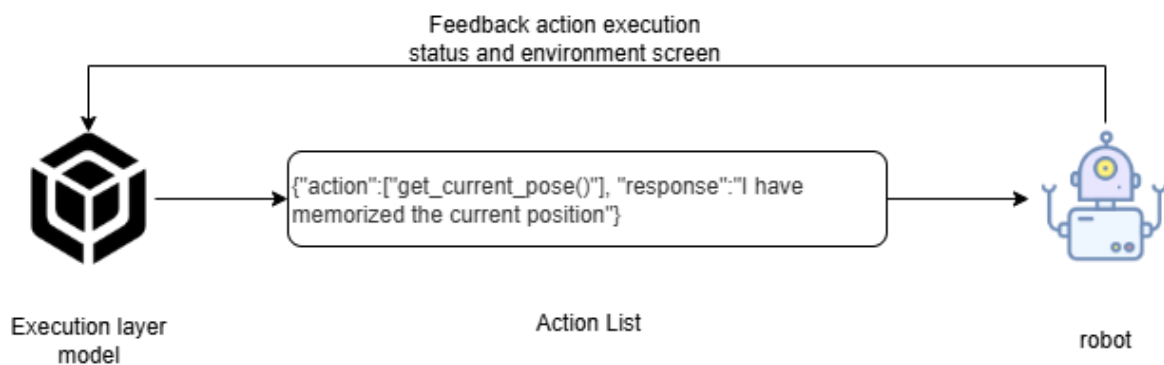
3.2 Decision Layer AI

- The decision layer large model is mainly responsible for task planning. It can understand complex human instructions and break them down into specific task steps. For example, upon receiving the command "Can you go to the kitchen and get me a bottle of mineral water?", the large language model will break it down into several tasks, as shown in the figure below.



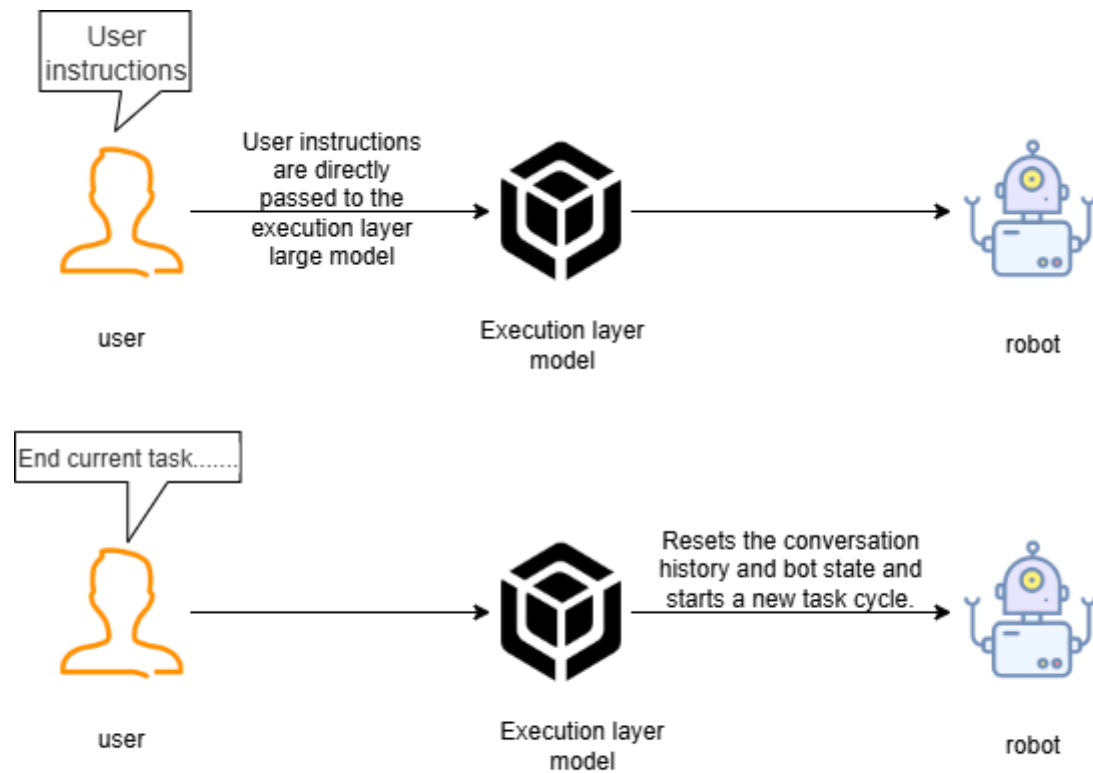
3.3 Execution Layer AI

- The execution layer large model is mainly responsible for chat responses and converting task steps into JSON text. It continuously receives feedback (success/failure) and images from the robot's action execution and provides the next action instruction based on the outcome.
- The execution layer large model acts as a supervisor, constantly monitoring the progress of the robot's task execution and deciding the next action based on feedback and environmental information, until the task is successfully completed or terminated early due to special circumstances.



4. Task Cycle

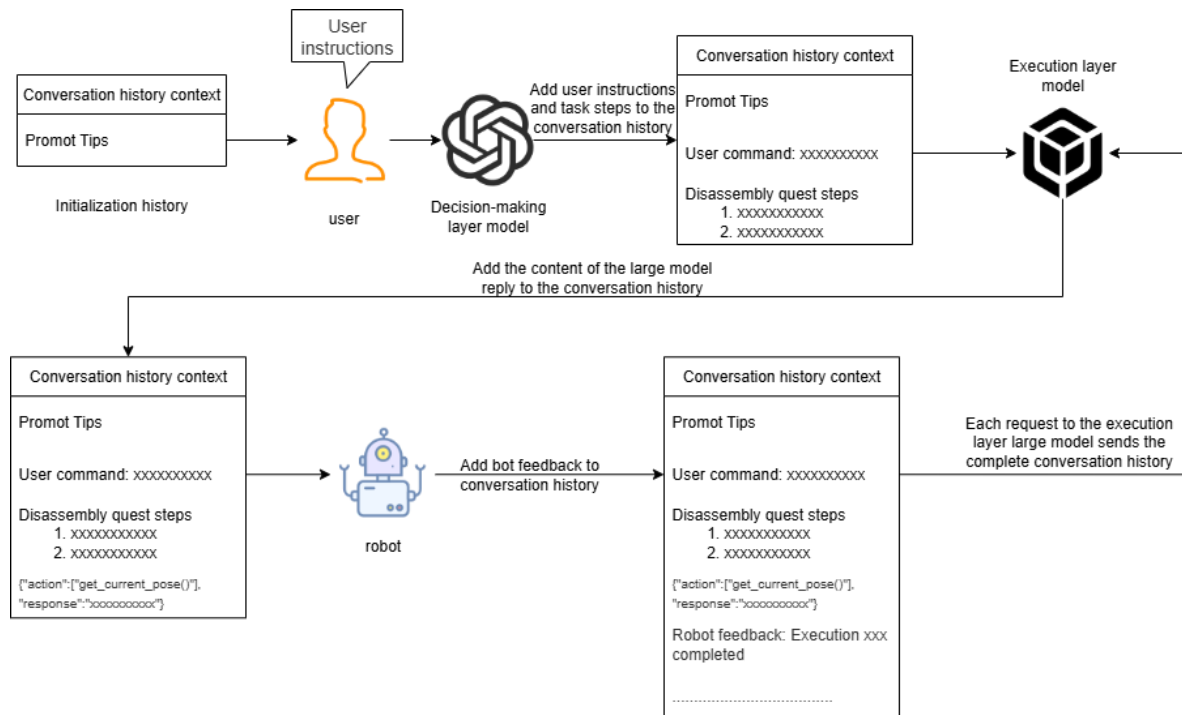
- After waking up the robot, a task cycle begins. All user commands and robot feedback are stored in the AI model's historical context, acting as the robot's temporary memory, allowing it to "remember" what happened before.
- When the user asks to end the current task and let the robot rest, the robot resets the AI model's historical context, causing it to have "amnesia" and forget what happened previously.



The waiting state of the robot after completing the task

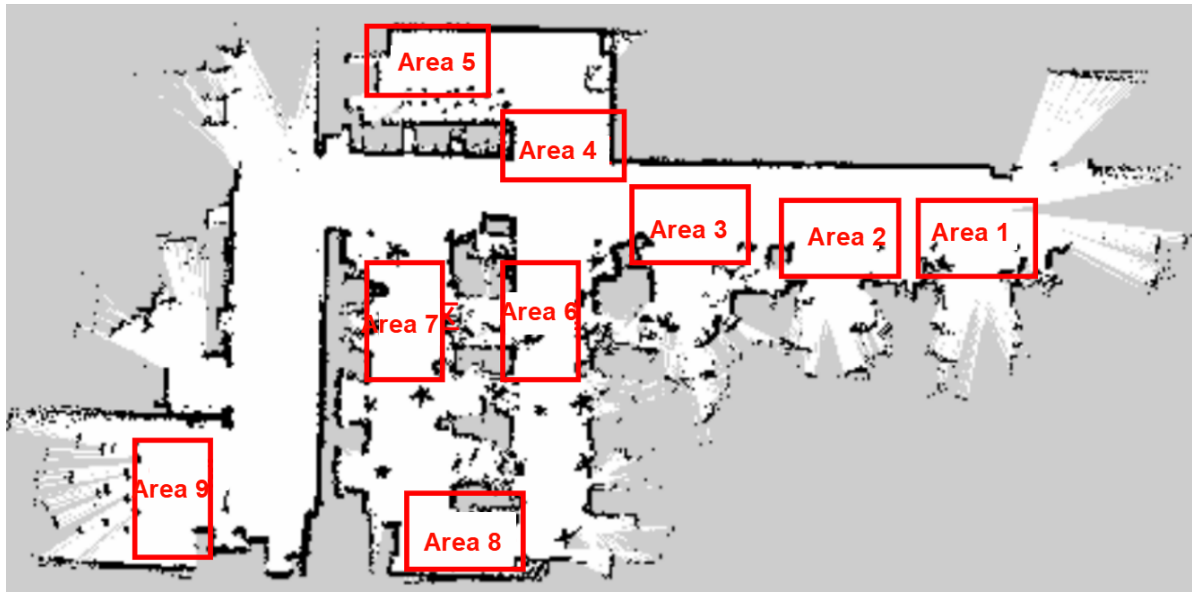
5. Historical Context

- The robot's conversation history is stored in a Dify AI application variable, with a default maximum memory of 50 conversation turns.



6. Map Mapping

- The robot uses a grid map for navigation. To make the robot understand real-world locations and areas, a mapping relationship must be established between the grid map and the real environment areas. This is called **map mapping**.
- Suppose we use a robot to generate a **grid map** in a factory environment using SLAM. The real factory has several manually divided areas, as shown in the figure below.



We establish a one-to-one correspondence between these real-world areas and letter symbols.

A: "Area 1", B: "Area 2", C: "Area 3", D: "Area 4", E: "Area 5", F: "Area 6", G: "Area 7", H: "Area 8", I: "Area 9"

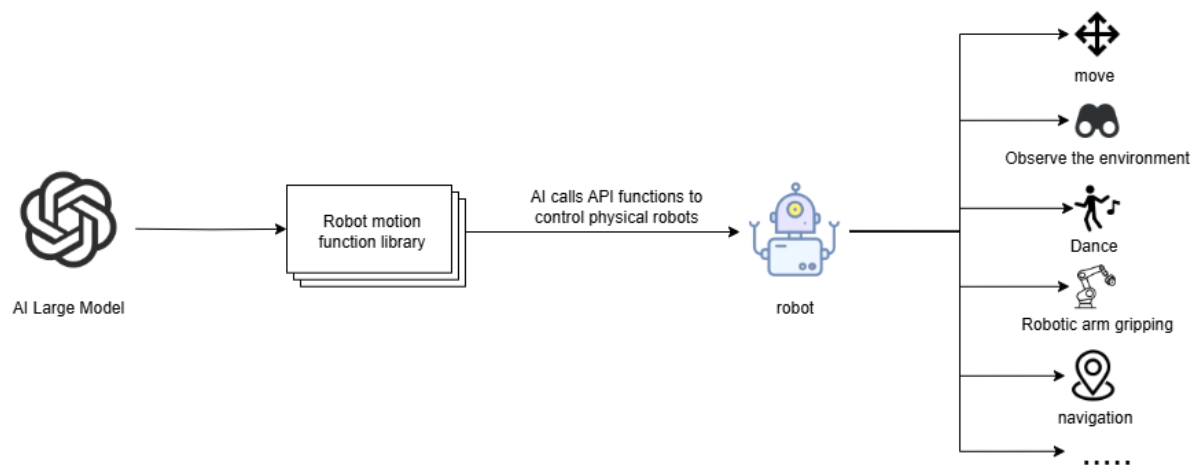
Then, we write the map coordinates for these letter symbols in a YAML file, for example:

```
A:
  name: 'Area 1'
  position:
    x: 4.4034953117370605
    y: 0.4879316985607147
  orientation:
    x: 0.0
    y: 0.0
    z: 0.701498621044694
    w: 0.7126708108744126
```

When we ask the robot to go to a specific real-world area, the large model just needs to convert it to the corresponding letter symbol, and the robot can then understand the location in the real environment.

7. Action Function Library

- The API functions in the robot's action function library are the bridge for the large model to control the robot's interaction with the real world.
- These API functions define the minimum actions that the physical robot can perform in the physical world.
- The AI large model sends the actions to be executed to the robot by converting them into JSON text, which the robot then parses to execute the corresponding actions.



All minimum action functions and their corresponding functionalities are shown in the table below:

Function Name	Parameters	Functionality	Example Command Reference
<code>move_left(x, angular_speed)</code>	x: rotation angle, angular_speed: angular velocity	Turn left by x degrees	Turn left x degrees
<code>move_right(x, angular_speed)</code>	x: rotation angle, angular_speed: angular velocity	Turn right by x degrees	Turn right x degrees
<code>dance()</code>	-	Trigger the robot's preset dance routine	Dance, perform a dance, etc.
<code>set_cmdvel(linear_x, linear_y, angular_z, duration)</code>	linear_x: x-axis linear velocity, linear_y: y-axis linear velocity, angular_z: z-axis angular velocity, duration: topic publishing duration	Control the robot chassis movement by setting linear and angular velocities	Move forward, backward, strafe left/right
<code>navigation(x)</code>	x: symbol corresponding to the target point in the map mapping	Control the robot to navigate to the target point	Navigate to a location
<code>navigation(zero)</code>	zero: a previously recorded coordinate point	Navigate back to the last recorded coordinate point	Return to origin, starting position
<code>get_current_pose()</code>	-	Record the current coordinates in the global map to the 'zero' parameter	Remember the current location
<code>track(x1, y1, x2, y2)</code>	(x1, y1, x2, y2): coordinates of the top-left and bottom-right points of the target object's bounding box	Track a specified object within the field of view	Track xx
<code>seewhat()</code>	-	Take a picture from the robot's current perspective and upload it to the execution layer large model	Observe the environment

Function Name	Parameters	Functionality	Example Command Reference
<code>follow_line()</code>	-	Move along a colored line on the ground	Start line following
<code>finish_dialogue()</code>	-	Reset the historical context, start a new task cycle	-
<code>finish()</code>	-	Automatically called after the robot completes a task to stop feeding back status to the execution layer model	-
<code>control_sign(mode)</code>	mode: enable, disable	Used to enable or disable the response to traffic signs in the AI-assisted autonomous driving case	Ignore traffic lights and signals en route
<code>alarm(mode)</code>	mode: on, off	Used to control whether the alarm function is enabled in the autonomous driving case	Start alarm
<code>get_dist(x1,y1,x2,y2)</code>	(x1,y1,x2,y2): coordinates of the top-left and bottom-right points of the target object's bounding box	Locate an object via the large model and measure the distance to any target	Observe the distance to xx in front of you
<code>control_light(mode)</code>	mode: ambient light mode, 0: off, 101: flowing, 102: marquee, 103: breathing, 104: gradient, 105: starlight	AI large model controls the mode of the front ambient light	Turn on flowing light

